Test 1

Sie können maximal 100 Punkte erreichen
Es sind zur Prüfung zugelassen:
* Transparente Wasserflasche
* Von mir ausgegebenes Papier und von Ihnen mitgebrachtes Schreibwerkzeug.

Alles andere ist nicht zugelassen.

- Viel Erfolg!

Notenschlüssel:
$[0-50\%)$: N5; $[50-62.5\%)$: G4; $[62.5-75\%)$: B3; $[75-87.5\%)$: G2; $[87.5-100\%]$: S1;

Aufgabe 01 - Eingabevalidierung für Muster - 60%

1. Funktionen der Eingabevalidierung

- **LeseFarbEmoji:** 🙄
  - Nimmt einen Userprompt als `Eingangsparameter` entgegen und erlaubt dem *Benutzer*, ein gültiges Symbol einzugeben.
  - ***Gültig bedeutet:*** Fragt den Benutzer in einer `Schleife` solange nach einer Eingabe, bis eines der vier erlaubten Farb-Quadrat-Emojis eingegeben wurde: 🟦, 🟩, 🟨 **oder** 🟥.
  - **Bei jeder anderen Eingabe** wird die Konsole mit `Console.Clear()` geleert, die Fehlermeldung ❌ `Ungültiges Symbol! Erlaubt sind nur: 🟦, 🟩, 🟨 oder 🟥.` ausgegeben und der Prompt erneut abgefragt.
- **LesePositiveGeradeZahl:** 🤔
  - Nimmt einen Userprompt als `Eingangsparameter` entgegen und erlaube dem *Benutzer* eine gültige Zahl einzugeben.
  - ***Gültig bedeutet:*** Verwendet eine `Schleife` in Kombination mit `int.TryParse`, um sicherzustellen, dass der Benutzer eine gerade, ganze Zahl eingibt, die **größer als 0** ist.
  - **Bei jeder anderen Eingabe** wird die Konsole mit `Console.Clear()` geleert, die Fehlermeldung ❌ `Ungültige Eingabe! Bitte eine positive gerade, ganze Zahl (> 0) eingeben.` ausgegeben und erneut gefragt.
- **LeseLaengeFuerDreierMuster:** 💀
  - Nimmt einen Userprompt als `Eingangsparameter` entgegen und erlaubt dem *Benutzer*, eine gültige Länge des Arrays einzugeben.
  - ***Gültig bedeutet:*** Liest die Eingabe in einer `Schleife` ein und überprüft nacheinander **drei mögliche ungewünschte Zustände (Fehlerfälle)** mit jeweils `Console.Clear()`, einer **eigenen, spezifischen Fehlermeldung**:
    a. **Fehlerfall 1 (Keine ganze Zahl):** Wenn die Umwandlung mit `int.TryParse` fehlschlägt, gib aus:
    > ❌ `Fehler: Ungültige Eingabe! Bitte geben Sie eine gültige ganze Zahl ein.`

b. **Fehlerfall 2 (Zahl ist kleiner als 3):** Wenn die eingegebene Zahl kleiner als 3 ist (`laenge < 3`), gib aus:

> ❌ `Fehler: Die Zahl ist zu klein! Die Länge muss mindestens 3 betragen.`

c. **Fehlerfall 3 (Zahl ist nicht durch 3 teilbar):** Wenn bei der Division durch 3 ein Rest bleibt (`laenge % 3 != 0`), gib aus:

> ❌ `Fehler: Ungültige Länge! Die Zahl muss ohne Rest durch 3 teilbar sein (z. B. 3, 6, 9, 12).`

- Erst wenn keiner der drei Fehlerfälle zutrifft (gewünschter Zustand), wird die Zahl (`laenge`) mit `return` zurückgegeben.

> Hinweis: Schreibe eine `Schleife` mit einer `Bedingung` die immer *true* ist. Verwende dann `Bedingte Anweisungen` um spezielle Fehler ausgeben zu können. Innerhalb der `Bedingten Anweisungen` für ungewünschte Zustände ❌ verwende anstatt *return* das `Keyword` ***continue*** um wieder zum Anfang der `Schleife` zu springen.

2. Funktionen für Muster

- `ZeichneZentriertesKreuz` 🧐:
  - Nimmt die Breite (**anzahlSpalten**), die Höhe (**anzahlZeilen**), das Symbol für das Kreuz (**kreuzSymbol**) sowie das Symbol für den Hintergrund (**hintergrundSymbol**) als `Eingangsparameter` entgegen und gibt das Muster direkt in der `Konsole` aus.
  - **Vorgehensweise:**
    - Zu Beginn stellt eine `Guard Clause` sicher, dass sowohl die Anzahl der Zeilen (**anzahlZeilen**) als auch die Anzahl der Spalten (**anzahlSpalten**) jeweils eine **gerade Anzahl** haben und größer als 0 ist. Falls eine der beiden Dimensionen ungerade ist, wird eine Fehlermeldung ausgegeben und die Methode sofort per `return` abgebrochen.
    - Es wird geprüft, ob wir uns auf den **beiden mittleren Zeilen** (**gibKreuzSymbolInZeileAus**) oder den **beiden mittleren Spalten** (**gibKreuzSymbolInSpalteAus**) befinden. Wenn ja, wird **kreuzSymbol** ausgegeben, andernfalls **hintergrundSymbol**.
- `ErstelleMuster` 🧐:
  - Nimmt die geprüfte Gesamtlänge (**laenge**, durch 3 teilbar) sowie zwei Farb-Emojis als `Eingangsparameter` entgegen (**doppelSymbol** und **einzelSymbol**).
  - Lege ein `string[] - Array` (**muster**) in der übergebenen Länge (**laenge**) an und befüllt es fortlaufend in Muster: In jedem Muster wird **zweimal das erste Symbol (doppelSymbol)** und danach **einmal das zweite Symbol (einzelSymbol)** eingetragen.
- `PrintArray` 😊:
  - Nimmt ein eindimensionales `string[] - Array` (**items**) entgegen und gibt alle Elemente mit einer `for` - Schleife im folgenden Format in einer Zeile aus:

    🚀>[🟦🟦🟧🟦🟦🟧]<🏁

- 🌟 **Bonusaufgabe** 💀:
  - Passe die Methode `ZeichneZentriertesKreuz` so an, dass sie **optional** auch mit **ungeraden** Zeilen- und Spaltenanzahlen umgehen kann:
  - Ist die Gesamtanzahl der Zeilen (**anzahlZeilen**) oder Spalten (**anzahlSpalten**) **ungerade**, gibt es keine zwei mittleren Zeilen/Spalten, sondern exakt **eine** mittlere Zeile bzw. Spalte.

3. Erwarteter Output

```
Bitte Symbol für das Kreuz eingeben (🟦, 🟩, 🟧, 🟥): 🟥
Bitte Symbol für den Hintergrund eingeben (🟦, 🟩, 🟧, 🟥): 🟩
Bitte gerade Breite X (> 0) eingeben: 4
Bitte gerade Höhe Y (> 0) eingeben: 6

--- 2D-Kreuz (4 x 6) ---
🟩🟩🟥🟥🟩🟩
🟥🟥🟥🟥🟥🟥
🟥🟥🟥🟥🟥🟥
🟩🟩🟥🟥🟩🟩

--- 1D Signal-Muster (2x Symbol A, 1x Symbol B) ---
Bitte 1. Signal-Symbol eingeben (wird 2x wiederholt): 🟦
Bitte 2. Signal-Symbol eingeben (wird 1x gesetzt): 🟧
Bitte Array-Länge eingeben (mind. 3 und durch 3 teilbar): 15

Ausgabe des geklonten Signal-Arrays:
🚀>[ 🟦🟦🟧🟦🟦🟧🟦🟦🟧🟦🟦🟧🟦🟦🟧 ]<🏁
```

Aufgabe 02 - Verschachtelungen umschreiben für Zugsuche ab Wien - 40%

1. Ausgangscode

Schreibe die in der `Funktion` *SuchePassendeVerbindung* befindliche `Verschachtelung` in eine `Gaurd Clause` um.

2. Referenztypen mit Werttypsemantik zuweisen, Guard Clauses und Suchfunktionen

1. `Arrays` **klonen im Hauptprogramm:** 🙂
   - Erstelle im Hauptprogramm vor dem Aufruf von `SuchePassendeVerbindung` kopien von (**ziele**, **zuege**, **fahrtzeitenMin**) der vorgegebenen Original- `Arrays` (**originalZiele**, **originalZuege**, **originalFahrtzeitenMin**), indem du direkt `.Clone()` mit dem passenden `Cast` (`(string[])` bzw. `(int[])`) aufrufst. Schreibe dazu den `Cast` vor den `Namen` der `Variable` .
2. `Guard Clauses` **in** `SuchePassendeVerbindung` **:** 🙂
   - Schreibe die `Verschachtelung` in `SuchePassendeVerbindung` zu `Guard Clauses` um. Tu dies innerhalt dieser `Funktion` .
   - Rufe für **ziel** `"Stadlau"` die Funktion `FindeSchnellsteVerbindung` auf und gib den ermittelten Zug aus **zuege** sowie die Fahrtzeit aus **fahrtzeiten** aus.

- Rufe für die Landeshauptstädte die Funktion `FindeGemuetlichsteVerbindung` auf und gib den ermittelten Zug aus **zuege** sowie die Fahrtzeit aus **fahrtzeiten** aus. Verwende dazu `Interpolated Strings` welche foglendermaßen zu schreiben sind `$"Text {variable}"` .

3. **`FindeSchnellsteVerbindung` 🤔:**
   - Nimm das gesuchte Ziel (**ziel**), das `Array` der Ziele (**ziele**) und das `Array` der Fahrtzeiten (**fahrtzeiten**) als `Eingangsparameter` entgegen und gibt den `Index` (**besterIndex**) als `int` zurück.
   - Durchsucht das `Array` **ziele** mit einer `Zählschleife` nach allen Einträgen, die mit **ziel** übereinstimmen, und ermittele den `Index` der **kürzesten Fahrtzeit**.
4. **`FindeGemuetlichsteVerbindung` 🤔:**
   - Wie `FindeSchnellsteVerbindung` , nur berechne die **längste Fahrtzeit**.

3. Erwarteter Output

```
🌄 Gemütlichste Verbindung von Wien nach Salzburg:
   Zug:       WB 910 (Index 2)
   Fahrtzeit: 215 Minuten

⚡ Schnellste Verbindung von Wien nach Stadlau:
   Zug:       S80 B (Index 28)
   Fahrtzeit: 9 Minuten

🌄 Gemütlichste Verbindung von Wien nach Linz:
   Zug:       WEST 908 (Index 7)
   Fahrtzeit: 128 Minuten

🌄 Gemütlichste Verbindung von Wien nach Graz:
   Zug:       RJ 533 (Index 12)
   Fahrtzeit: 220 Minuten

🌄 Gemütlichste Verbindung von Wien nach Innsbruck:
   Zug:       ICE 1204 (Index 21)
   Fahrtzeit: 345 Minuten
❌ Fehler: Unbekanntes Ziel! Erlaubt sind nur Linz, Salzburg, Graz, Innsbruck oder Stadlau.
❌ Fehler: Der Startbahnhof muss immer 'Wien' sein!
```